

Sanitizing Inputs to Protect LLMs Against Prompt Injection Attacks

*A pipeline for LLM prompts that protects
against instructions in user-supplied data.*

Washington State University

Voiland College of Engineering and Architecture

School of Electrical Engineering and Computer Science

Scalable Algorithms for Data Science Lab



CyberCougs SCADS

Ryan Brombaugh, William Fralia, and Emily West

I. Introduction

Prompt injection is among the top ten concerns for large language models and generative artificial intelligence according to the OWASP GenAI Security Project [1]. Our project aims to address this issue by concentrating on mitigating **prompt injection** through data sources. This will culminate into a three step pipeline where we can test mitigation techniques. These steps include a web-based graphical user interface for input of a prompt and data, a sanitation step to reformat the original prompt and data which aims to prevent **prompt injection** from the data, and finally the construction of a new prompt that will be fed into various large language models (LLMs) for testing purposes. Our project assumes the user is not malicious, and any **prompt injection** attempts would be only present within the data portion of the prompt.

II. Background and Related Work

LLM prompt injection is a category of attacks where an adversary manipulates the inputs to an **LLM** such that it responds with unintended results. Depending on the scope and capabilities of the **LLM**, these attacks may enable an adversary to access sensitive information or trigger unintended or malicious actions within a system. This manipulation can occur directly through malicious user prompting, but as more robust systems integrate **LLMs** and rely on techniques like Retrieval-Augmented Generation (RAG) to connect **LLMs** to APIs, documents, or other resources, new potential attack vectors for **prompt injection** are added [2]. This project will focus on techniques to secure retrieved resources from **prompt injection** by creating a system that simulates prompting a user instruction along with data separately, running the instruction and data through sanitation techniques, and prompting various **LLMs** with the sanitized output. It is important that the sanitation mitigates any overriding of user input while still maintaining the usefulness of the non-malicious data after sanitization.

SecAlign is a research project that similarly focuses on mitigating **prompt injections** from external sources [2]. The work utilizes **preference optimization** to train **LLMs** to identify secure outputs over insecure outputs. This involves creating a dataset of different outputs for a given prompt-injected input and labeling them secure or insecure. Through **preference optimization** they lowered the probability of the unintended outputs over the secure outputs.

Experimental results show that this method is effective, reducing the success rate of various **prompt injection attacks** to below 10%. However, despite achieving stronger performance than prior approaches, the use of **preference optimization** introduces significant training cost and complexity, requiring large amounts of preference data generation.

DRIP is another research project that addresses this problem with two combined approaches [3]. The project created a system to mitigate **prompt injection** by separating any external data into “instruction” and “data” tokens. It then removed the “instruction” tokens while preserving the data. The project additionally includes a minimal residual module that prevents the external data from modifying the original user instruction by a major factor.

This research project’s approach of editing tokens allows for maintaining utility of data that may include **prompt injection**. The token identification and modification techniques within the project are valuable for our team to consider while we craft our own natural language processing solution.

Our team’s project aims to build on the current solutions to **prompt injection** by identifying current weaknesses to modern approaches of **prompt injection** security and experimenting to

find solutions using natural language processing and machine learning. As a team we will need to do further research on the current solutions to **prompt injection** and identify where our project can fill the gaps. We will need a strong grasp on natural language processing techniques as well as identifying instruction and data semantics. This will involve learning new Python libraries and working with different **LLMs** to build out our pipeline. Additionally, our team must research prompt structuring and **prompt injection** techniques to better understand our problem and build a reliable solution

III. Project Overview

The increase in **LLM** use in professional contexts introduces a new **LLM prompt injections** attack vector into internal systems. Areas such as the military, recruiting firms, and universities are highly at risk as organizations that take in considerable amounts of data, often from the public Internet at large.

At this time, current research is promising in developing systems that detect and sanitize user-supplied data, and construct safe data to provide with an LLM. However, limiting gaps remain, such as high compute costs for training, and space costs for storing large amounts of input and output pairs. This project aims to develop a program built upon this current research while reducing the amount of compute and space required to produce an effective sanitization program.

The overall goal of this project is a complete pipeline with a Graphical User Interface (GUI) for user input, a **data sanitization** engine, **prompt hardening** and reconstruction, and finally a call to an **LLM** via API that returns a safe output back to the user GUI. Therefore, the project is modularized into three parts. We expect the **data sanitization** part to be the most expensive in terms of time and effort.

The key milestones and goals of this project are outlined below:

- **Initial Research and Learning:** The team will research existing **prompt injection** sanitization projects, styles of **prompt injections**, and natural language processing APIs.
- **Data Collection:** The team will research datasets of **prompt injections** on HuggingFace to obtain real-world data to test the sanitization engine on and to discover patterns, keywords, and phrases common to **prompt injection**.
- **Implementation of Data Sanitization:** The team will develop **prompt injection** patterns and key phrases to detect, then use Python packages such as Natural Language Toolkit (NLTK) to implement tokenization, phrase detection, and masking.
- **Prototype Development:** To finish the initial prototype, the team will develop a simple web interface for the user and the prompt reconstruction process involving an API call to an LLM.
- **Testing and Optimization:** The team will use datasets of **prompt injections** and benign instructions to test the **data sanitization** engine. This will help to identify edge and corner cases to help improve the model's robustness.
- **Implement Stretch Goal:** The team will implement stretch goals such as making several **LLMs** available to the user in the GUI webpage.
- **Documentation and Final Presentation:** The team will discuss the stack, algorithms and techniques used in the project, its performance, and areas of future development. The team will showcase the project in a final showcase to demonstrate the project and its key takeaways.

IV. Client and Stakeholder Identification and Preferences

Our primary client is Tashi Stirewalt, a WSU cybersecurity PhD student advised by Professor Assefaw Gebremedhin. Both contribute to the VICEROY (Virtual Institutes for Cyber and Electromagnetic Spectrum Research and Employ) program, a federal program to promote cybersecurity education and research in universities. An intermediary goal of Professor Gebremedhin for this project is a working prototype to submit to the VICEROY Symposium poster session at William and Mary in Williamsburg, Virginia from April 14-16, 2026 [4]. If CyberCougs SCADS decides to move forward with this, reaching that goal will involve frequent meetings with Tashi, the primary contact for at least two weeks until the poster draft is submitted.

On a technical level, another goal of our clients is to research and build upon existing techniques for **prompt injection** detection. For example, one existing solution performs best when **prompt injection** is placed at the end of user-supplied data, and others do not perform prompt reconstruction for further system hardening.

As VICEROY is a federally funded program, the primary stakeholders or customers of this project would be the federal government, where input data to an **LLM** needs to be distrusted and heavily scrutinized by an intermediary system. This way, internal government systems can have a higher level of confidence in data that is being brought into secure systems. This project would support other ongoing efforts to make **LLMs** accessible in secure government areas, such as airgapped **LLMs** in dedicated servers.

Other stakeholders are in the private sector with similar needs for defense against distrusted incoming data. On a smaller scale, customers include university researchers and hiring managers searching for candidates on websites like LinkedIn. These pose threats to inject small but damaging scripts that could promote one candidate over another or infiltrate a university's network.

V. Glossary

Define technical terms used in the document.

LLM - Large Language Model. A type of machine learning mode that specializes in receiving prompts in natural language. Examples include ChatGPT, Claude, and Deepseek.

Prompt Injection – A cyberattack against **LLMs** in which malicious inputs are disguised as legitimate prompts.

PIA – Prompt injection Attacks. Attacks use **Prompt Injection** in an attempt to make a **LLM** act in an undesirable way.

Regular Expression (Regex) – A specialized sequence of characters forming a search pattern.

Prompt Hardening – Securing the user's prompt against outside interference.

Preference Optimization – Aligning the model to the current user's preferences to allow for better responses.

Data Sanitization – Cleaning user input to prevent accidental or purposeful manipulation of the systems meant to process the prompt.

VI. References

- [1] OWASP LLM Project Admin, "OWASP Top 10: LLM & Generative AI Security Risks," OWASP Top 10 for LLM & Generative AI Security, Nov. 18, 2024. <https://genai.owasp.org/>

- [2] S. Chen, "SecAlign: Defending Against Prompt Injection with Preference Optimization," *SIGSAC Conference on Computer and Communications Security*, vol. CCS '25, Nov. 2025. doi:<https://dl.acm.org/doi/10.1145/3719027.3744836>

- [2] "Facebookresearch/SECALIGN: Repo for the Research Paper 'SECALIGN: Defending against Prompt Injection with Preference Optimization.'" *GitHub*, github.com/facebookresearch/SecAlign. Accessed 6 Feb. 2026.

- [3] R. Liu, Y. Lin, Z. Huang, and J. S. Dong, "DRIP: Defending prompt injection via token-wise representation editing and residual instruction fusion," *arXiv.org*, <https://arxiv.org/abs/2511.00447> (accessed Feb. 6, 2026).

- [4] "Symposium - VICEROY Scholars," *Viceroy Scholars*, Mar. 27, 2024. <https://www.viceroy scholars.org/symposium/> (accessed Feb. 07, 2026).